OUTPUT:

```
PS C:\Users\ADMIN\Downloads\NNDL-2026\Experiment-6_LSTM> python .\LSTM.PY
Vocab size: 63
[',', '.', 'and', 'artificial', 'based', 'can', 'changing', 'complex', 'computers', 'continuously']

Epoch: 0, Loss: 4.1514
Epoch: 2, Loss: 4.1038
Epoch: 4, Loss: 4.0516
Epoch: 6, Loss: 3.9887
Epoch: 8, Loss: 3.9067
Epoch: 10, Loss: 3.7934
Epoch: 12, Loss: 3.6346
Epoch: 14, Loss: 3.4414
Epoch: 16, Loss: 3.2762
Epoch: 18, Loss: 3.0109

machine learning helps computers learn . data . states continuously learn intelligent networks solve data . recurrent the continuously . the . data the data helps learning learn problems la
nguage helps language data learning on
```

CODE:

```python
import torch
import torch.nn as nn

text = """
Artificial intelligence is changing the world.
Machine learning helps computers learn from data.
Deep learning uses neural networks to solve complex problems.
Recurrent neural networks process sequential information.
They remember information from previous steps using hidden states.
Natural language processing helps machines understand human language.
Text generation predicts the next word based on previous words.
Learning requires data, patience, and repeated experiments.
Technology can help people solve important problems.
Researchers continuously improve intelligent systems.
""" * 20

text = text.lower().replace(".", " .").replace(",", " ,")
words = text.split()

vocab = sorted(set(words))
word_to_id = {w: i for i, w in enumerate(vocab)}
id_to_word = {i: w for i, w in enumerate(vocab)}
vocab_len = len(vocab)

print("Vocab size:", vocab_len)
print(vocab[:10])

encoded = torch.tensor([word_to_id[w] for w in words])

seq_length = 5
X = []
Y = []

for i in range(0, len(encoded) - seq_length):
    X.append(encoded[i : i + seq_length])
    Y.append(encoded[i + seq_length])

X = torch.stack(X)
Y = torch.stack(Y)
X = nn.functional.one_hot(X, num_classes=vocab_len).float()


class MyLSTM(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size
        n = input_size + hidden_size

        self.W_f = nn.Parameter(torch.randn(n, hidden_size) * 0.1)
        self.b_f = nn.Parameter(torch.ones(hidden_size))
        self.W_i = nn.Parameter(torch.randn(n, hidden_size) * 0.1)
        self.b_i = nn.Parameter(torch.zeros(hidden_size))
        self.W_g = nn.Parameter(torch.randn(n, hidden_size) * 0.1)
        self.b_g = nn.Parameter(torch.zeros(hidden_size))
        self.W_o = nn.Parameter(torch.randn(n, hidden_size) * 0.1)
        self.b_o = nn.Parameter(torch.zeros(hidden_size))

    def forward(self, X):
        B, T, _ = X.shape
        h = torch.zeros(B, self.hidden_size)
        c = torch.zeros(B, self.hidden_size)

        for t in range(T):
            x = X[:, t, :]
            xh = torch.cat([x, h], dim=1)

            f = torch.sigmoid(xh @ self.W_f + self.b_f)
            i = torch.sigmoid(xh @ self.W_i + self.b_i)
            g = torch.tanh(xh @ self.W_g + self.b_g)
            o = torch.sigmoid(xh @ self.W_o + self.b_o)

            c = f * c + i * g
            h = o * torch.tanh(c)

        return h


class LSTMText(nn.Module):
    def __init__(self, vocab_len, hidden_size):
        super().__init__()
        self.lstm = MyLSTM(vocab_len, hidden_size)
        self.fc = nn.Linear(hidden_size, vocab_len)

    def forward(self, X):
        h = self.lstm(X)
        return self.fc(h)


model = LSTMText(vocab_len, 128)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.003)

for epoch in range(20):
    prediction = model(X)
    loss = criterion(prediction, Y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 2 == 0:
        print(f"Epoch: {epoch}, Loss: {loss.item():.4f}")

def generate_text(model, seed, length=30, temperature=0.8):
    model.eval()
    generated = seed.lower().split()

    with torch.no_grad():
        for _ in range(length):
            context = generated[-seq_length:]
            idx = torch.tensor([[word_to_id[w] for w in contex
            x = nn.functional.one_hot(idx, num_classes=vocab_l

            logits = model(x) / temperature
            probs = torch.softmax(logits, dim=-1)

            next_id = torch.multinomial(probs, num_samples=1).
            generated.append(id_to_word[next_id])

    return " ".join(generated)


seed = "machine learning helps computers learn"
print(generate_text(model, seed, length=30))
```